

and

$$x_i \geq 0, \quad i = 1, \dots, n + m \quad (10.10.18)$$

Here we assume that we started with an $m \times n$ matrix of constraint coefficients given by equations like (10.10.3) and (10.10.4). We then added slack variables to the inequality constraints and artificial variables to the equality constraints so that the constraint matrix is now the $m \times (n + m)$ matrix $\mathbf{A}$. The last m columns form an $m \times m$ identity matrix. Note that the coefficients of the slack variables are taken to be +1, so that an original $\geq$ inequality will have a negative right-hand side. For our example given in equations (10.10.5) – (10.10.8), transformed as in equations (10.10.9), (10.10.10), and (10.10.14), the matrix $\mathbf{A}$ has five columns:

$$\mathbf{a}_1 = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ -1 \\ 3 \end{pmatrix} \quad \mathbf{a}_3 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \mathbf{a}_4 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \mathbf{a}_5 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \quad (10.10.19)$$

The right-hand side and the objective function coefficients are

$$\mathbf{b} = \begin{pmatrix} 70 \\ -40 \\ 90 \end{pmatrix} \quad \mathbf{c} = (-40, -60, 0, 0, 0)^T \quad (10.10.20)$$

We partition the matrix $\mathbf{A}$ into two submatrices,

$$\mathbf{A} = [\mathbf{A}_B \mid \mathbf{A}_N] \quad (10.10.21)$$

where we have permuted the columns corresponding to the basic variables to be in $\mathbf{A}_B$, while the nonbasic columns are in $\mathbf{A}_N$. In our example, the initial basic variables are (x_3, x_4, x_5) and the initial basis $\mathbf{A}_B$ is the unit matrix composed of the last three columns of $\mathbf{A}$, $\mathbf{a}_3$, $\mathbf{a}_4$ and $\mathbf{a}_5$. A basic solution of $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ consists of a set of basic and nonbasic variables $[\mathbf{x}_B \mid \mathbf{x}_N]$ with $\mathbf{x}_N = 0$. In our example, initially $\mathbf{x}_N = (x_1, x_2) = 0$. The basic solution satisfies $\mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$, or $\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}$.

To derive the simplex method, we need one simple equation: how a basic vector changes as a nonbasic variable (i.e., one that is zero) becomes nonzero. This corresponds to starting at a vertex of the simplex and sliding along an edge toward another vertex. Suppose the variable x_k is the one increasing from zero. The constraint equation $\mathbf{A} \cdot \mathbf{x} = \mathbf{A}_B \cdot \mathbf{x}_B = \mathbf{b}$ becomes

$$\mathbf{A}_B \mathbf{x}'_B + \mathbf{a}_k x_k = \mathbf{b} \quad (10.10.22)$$

since only x_k is nonzero among the nonbasic variables. Multiplying this equation by $\mathbf{A}_B^{-1}$ gives

$$\mathbf{x}'_B = \mathbf{A}_B^{-1} \cdot \mathbf{b} - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k = \mathbf{x}_B - x_k \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \quad (10.10.23)$$

The first application of equation (10.10.23) is to the idea of a *reduced cost*. The coefficient c_i in the objective function (10.10.16) is sometimes called the cost of variable x_i , because it represents the cost of having x_i amount of quantity i in the objective function. The simplex method requires instead the cost of *changing* a variable that is zero (not in the basis) to a nonzero value. If the initial value of the objective function is $\mathbf{c} \cdot \mathbf{x}_B = \mathbf{c}_B \cdot \mathbf{x}_B$ and the final value is $\mathbf{c} \cdot (\mathbf{x}'_B + x_k \mathbf{e}_k)$, where $\mathbf{e}_k$

is a unit vector, then using equation (10.10.23) you find that the difference is $x_k u_k$, where the reduced cost of x_k is given by

$$u_k = c_k - \mathbf{a}_k \cdot \mathbf{y}, \quad \mathbf{y} \equiv (\mathbf{A}_B^{-1})^T \cdot \mathbf{c}_B \quad (10.10.24)$$

Note that if $u_k < 0$, you can make the value of the objective function smaller by bringing x_k into the basis (making it nonzero).

The simplex procedure consists of the following steps:

1. Find a feasible basis (phase 1)
2. Compute the reduced costs (10.10.24) for all x_k not in the basis.
3. If $u_k \geq 0$ for all k , the solution is optimal: No exchange will improve things. Otherwise, choose k corresponding to the most negative u_k as the entering column.
4. Choose the leaving column i from the *minimum ratio test* (motivated below): Compute

$$\mathbf{x}_B = \mathbf{A}_B^{-1} \cdot \mathbf{b}, \quad \mathbf{w} = \mathbf{A}_B^{-1} \cdot \mathbf{a}_k \quad (10.10.25)$$

For each component $w_i > 0$, compute the ratio x_B^i / w_i . Choose i that corresponds to the smallest such ratio. (If no $w_i > 0$, the objective function is unbounded. Exit and report this.)

5. Exchange columns i and k and go back to step 2.

The minimum ratio test is the second application of equation (10.10.23), which can be written as

$$(x_B^i)' = x_B^i - x_k w_i \quad (10.10.26)$$

where w_i is defined in equation (10.10.25). For each $w_i > 0$, x_B^i decreases as x_k increases from zero. The minimum ratio test selects the i corresponding to the first x_B^i to hit zero, while the other basis variables are still positive. The idea is to allow x_k to be as big as possible so that the objective function is reduced as much as possible by bringing it into the basis.

Let's see how this applies to our example. We start with phase zero, where we remove the zero variable x_5 from the basis. Suppose we choose x_2 to be the incoming variable (x_1 would work fine, too). Using x_2 , we find for the new basis and its inverse

$$\mathbf{A}_B = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 3 \end{pmatrix} \quad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & 0 & -\frac{1}{3} \\ 0 & 1 & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} \end{pmatrix} \quad (10.10.27)$$

The new basic solution is

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_4 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 40 \\ -10 \\ 30 \end{pmatrix} \quad (10.10.28)$$

The solution (10.10.28) is not feasible because x_4 is negative. We enter phase one, with $\zeta' = \bar{\mathbf{c}} \cdot \mathbf{x} = -x_4$, i.e.,

$$\bar{\mathbf{c}}_B = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix} \quad (10.10.29)$$

Here the order of elements corresponds to the order (x_3, x_4, x_2) for the basic variables. We compute the reduced costs from equation (10.10.24). Only $k = 1$ is relevant, since x_5 is never allowed to re-enter the basis (zero variable). We find

$$u_1 = -\mathbf{a}_1 \cdot (\mathbf{A}_B^{-1})^T \cdot \bar{\mathbf{c}}_B = -\frac{2}{3} \quad (10.10.30)$$

which is negative, confirming that x_1 should enter. For the minimum ratio test to determine which variable leaves, we need the quantity

$$\mathbf{A}_B^{-1} \cdot \mathbf{a}_1 = \begin{pmatrix} \frac{5}{3} \\ -\frac{2}{3} \\ \frac{1}{3} \end{pmatrix} \quad (10.10.31)$$

So the ratios of the elements in equation (10.10.28) to those in equation (10.10.31) are

$$\frac{40}{5/3} = 24, \quad \frac{-10}{-2/3} = 15, \quad \frac{30}{1/3} = 90 \quad (10.10.32)$$

The middle ratio is the minimum, so x_4 goes out. (Note that in phase one we relax the requirement that $w_i > 0$, since we haven't yet made all the variables nonnegative.)

Now the basic variables are (x_3, x_1, x_2) . Proceeding as before, we find

$$\mathbf{A}_B = \begin{pmatrix} 1 & 2 & 1 \\ 0 & -1 & -1 \\ 0 & 1 & 3 \end{pmatrix} \quad \mathbf{A}_B^{-1} = \begin{pmatrix} 1 & \frac{5}{2} & \frac{1}{2} \\ 0 & -\frac{3}{2} & -\frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{pmatrix} \quad (10.10.33)$$

and

$$\mathbf{x}_B = \begin{pmatrix} x_3 \\ x_1 \\ x_2 \end{pmatrix} = \mathbf{A}_B^{-1} \cdot \mathbf{b} = \begin{pmatrix} 15 \\ 15 \\ 25 \end{pmatrix} \quad (10.10.34)$$

All the variables are positive, so the basis is feasible and we enter phase two, with $\mathbf{c}_B = (0, -40, -60)^T$. We find the reduced cost $u_4 = -30$, so x_4 re-enters the basis. The minimum ratio test (10.10.25) gives a minimum for the term involving x_3 , so the next basis is (x_4, x_1, x_2) . The basic solution turns out to be $(6, 24, 22)$. When we compute the reduced cost u_3 for this basis, it is positive, so we are done. The minimum occurs at $x_1 = 24$, $x_2 = 22$, and the minimum value, obtained by substitution in the objective function, is -2280 . The meaning of $x_4 = 6$ is that the inequality (10.10.7) is satisfied by 6. The other two constraints are satisfied as equalities.

The graphical interpretation of the solution procedure is shown in Figure 10.10.2. The initial basic vector is at the origin. We first proceed to the vertex A , which puts us on the line where the equality (10.10.8) is satisfied. This is not a feasible point, since we are on the wrong side of the line Y . So we move along the line X to the vertex B , which is now feasible. Finally we move to vertex C , which is the minimum value of the objective function.

10.10.4 Degeneracy

Nonbasic variables in a basic feasible solution are all zero. If any *basic* variable is zero, we say the basis is *degenerate*. Geometrically, this situation corresponds in n dimensions to having more than n hyperplanes intersect at a vertex. Degeneracy can cause problems in the simplex method. Consider the simple case when three

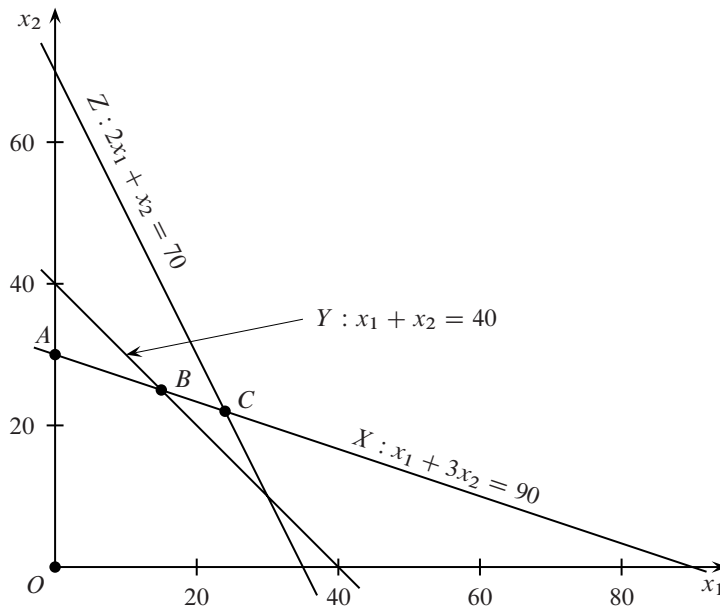


Figure 10.10.2. Graphical interpretation of the simplex solution of the problem (10.10.5) – (10.10.8). The initial basic vector is at the origin O . To satisfy the equality (10.10.8), the first step moves to A , on the line X . This is not yet a feasible point, since it is on the wrong side of the line Y . The next move is to B , which is feasible. We enter phase two, and find that we can reduce the objective function by moving to C . No further moves are possible, so we are done. Note that the figure is really the projection of a five-dimensional simplex onto the x_1 - x_2 plane.

lines intersect at a point in two dimensions. Only two of the lines are necessary to define the vertex. When the leaving variable is chosen, it can correspond to the third direction at the vertex. On making this change in the basis, the objective function doesn't improve. You can see this algebraically from equation (10.10.26): If $x_B^i = 0$ and $w^i > 0$, then a step size of zero is required for the new variable x_k . Clearly special measures need to be taken.

Degeneracy allows the possibility of *cycling*, where you keep exchanging the same set of vectors in and out of the basis without making any progress. In practice, however, cycling is almost never a problem. More common in very degenerate problems is *stalling*, where you spend a long time making exchanges before finally leaving the vertex.

10.10.5 Sparseness and Stability

If you examine the operations carried out during each simplex step, you see that a key ingredient is to solve equations of the form $\mathbf{A}_B \cdot \mathbf{x} = \mathbf{b}$ and similar equations with $\mathbf{A}_B^T$. We know that a good method for doing this, absent other considerations, is to use the LU decomposition of $\mathbf{A}_B$ (cf. §2.3), since we can use partial pivoting to maintain stability. Decomposing $\mathbf{A}_B$ afresh each step is expensive, but since successive bases differ only by the replacement of a single column, one can use techniques analogous to the Sherman-Morrison formula (§2.7) to update $\mathbf{L}$ and $\mathbf{U}$.

However, most linear programming problems that occur in practice have a constraint matrix $\mathbf{A}$ that is very sparse. It is crucial to take advantage of this sparseness in the linear algebra procedures that make up each simplex step, since real-life prob-

lems can involve many thousands or even millions of constraints and variables. Standard LU decomposition with partial pivoting to maintain stability is not desirable, because it leads to excessive *fill-in*, that is, the generation of nonzero matrix elements where there were zeros before. Instead, one chooses the pivot element by a trade-off between stability and sparsity. A popular strategy is based on the *Markowitz criterion* [4]. Here the pivot is a nonzero with the smallest product of the number of other nonzeros in its row and its column. Empirically, the Markowitz criterion works about as well as any other general strategy for minimizing fill-in. In linear programming applications, one also generally needs to impose some kind of stability criterion. In *threshold partial pivoting*, no pivot is less than α times the largest element in its row, where α is a parameter between zero and one. A typical value is $\alpha \sim 0.1$; $\alpha = 1$ gives straight partial pivoting.

The first stable updating procedure for sparse LU decomposition was given by Bartels and Golub [5,6]. This procedure updates $\mathbf{L}$ and $\mathbf{U}$ when a column is replaced in $\mathbf{A}_B$. A good sparse LU algorithm that includes the Bartels-Golub update is that of [7]. It is freely available in the software package LUSOL as part of [8], and we use it in our pedagogical implementation described below.

There are newer methods for the LU decomposition and the update procedure. According to [9], the best of these is probably that of [10].

10.10.6 State-of-the-Art Simplex Codes

A high-quality implementation of the simplex method will have a number of features that we have not discussed so far.

- It will implement more than one variant of the simplex method. In addition to the algorithm we have described, the *primal algorithm*, it will typically also implement the so-called *dual algorithm* (duality is discussed in §10.11). The number of iterations can be significantly reduced by the appropriate use of more than one algorithm.
- It will accept multiple input formats for the specification of the problem, with suitable error checking.
- It will preprocess the problem, with the aim of reducing its size and improving its numerical properties. Many complex problems are inadvertently specified in reducible form.
- It will have multiple options for scaling the problem. As with solving linear equations, no universal scaling algorithm is known for linear programming.
- It will have multiple options for starting the iteration, including several procedures for phase 1.
- It will have several pricing strategies (procedures for selecting the incoming variable). These go by names like multiple pricing, Devex, and steepest edge.
- It will have multiple pivot methods (variants of the ratio test) for the outgoing variable.
- It will handle bounded variables, that is, variables that satisfy a requirement $l_i \leq x_i \leq u_i$ instead of $x_i \geq 0$. It is possible to handle such bounds using slack variables, but that increases the size of the matrix $\mathbf{A}$. A slight generalization of the algorithm we have described allows you to handle bounded variables directly.
- It will have efficient sparse matrix algorithms.

All these issues are thoroughly discussed in [9].

There are several excellent public domain simplex implementations that incorporate most of the above items. These include CLP [11], GLPK [12] and `lp_solve` [8]. If you are planning to do any serious LP solving, you should definitely explore these options. It may even be worthwhile to invest in a commercial LP solver. For more information on all these options, see [13]. But first look at the next section, where *interior-point methods* are discussed. It appears now that for many very big problems (but not all), interior-point methods can out-perform simplex methods [14–16]. Even for moderate-sized problems, an interior-point method could be your best choice.

The simplex codes mentioned above are large software efforts, with many thousands of lines of code. They are quite formidable if you want to study how the simplex algorithm works and play around with various options. Accordingly, in a Webnote [17], we give a pedagogical implementation of the simplex method. Even though it uses reasonably good sparse matrix algebra, it is slower than the public domain implementations by one to two orders of magnitude on problems with $\sim 10^4$ variables. If you don't care about the simplex algorithm and you just want a simple method to get your problem solved quickly without getting a public domain code up and running, take a look at our pedagogical interior-point code in the next section.

10.10.7 Other Topics Briefly Mentioned

Problems where the objective function and/or one or more of the constraints are replaced by expressions nonlinear in the variables are called *nonlinear programming problems*. The literature on such problems is vast, but outside our scope. The special case of quadratic expressions is called *quadratic programming*. Optimization problems where the variables take on only integer values are called *integer programming problems*, a special case of *discrete optimization* generally. Section 10.12 looks at a particular kind of discrete optimization problem.

CITED REFERENCES AND FURTHER READING:

- Bland, R.G. 1981, "The Allocation of Resources by Linear Programming," *Scientific American*, vol. 244 (June), pp. 126–144.[1]
- Dantzig, G.B. 1963, *Linear Programming and Extensions* (Princeton, NJ: Princeton University Press).[2]
- Kolata, G. 1982, "Mathematician Solves Simplex Problem," *Science*, vol. 217, p. 39.[3]
- Markowitz, H.M. 1957, "The Elimination Form of the Inverse and Its Application to Linear Programming," *Management Science*, vol. 3, pp. 255–269.[4]
- Bartels, R.H., and Golub, G.H. 1969, "The Simplex Method of Linear Programming Using the LU Decomposition," *Communications of the ACM*, vol. 12, pp. 266–268.[5]
- Bartels, R.H. 1971, "A Stabilization of the Simplex Method," *Numerische Mathematik*, vol. 16, pp. 414–434.[6]
- Gill, P.E., Murray, W., Saunders, M.A. and Wright, M.H. 1987, "Maintaining LU Factors of a General Sparse Matrix," *Linear Algebra and Its Applications*, vol. 88/89, pp. 239–270.[7]
- `lp_solve`, http://groups.yahoo.com/group/lp_solve. [8]
- Maros, I. 2003, *Computational Techniques of the Simplex Method* (Boston: Kluwer).[9]
- Suhl, U., and Suhl, L. 1990, "Computing Sparse LU Factorizations for Large-Scale Linear Programming Bases," *ORSA Journal on Computing*, vol. 2, pp. 325–335; 1993, "A Fast LU Update for Linear Programming," *Annals of Operations Research*, vol. 43, pp. 33–47.[10]
- CLP, <http://www.coin-or.org/Clp>. [11]

- GLPK (GNU Linear Programming Kit), <http://www.gnu.org/software/glpk>. [12]
- Linear Programming Frequently Asked Questions, <http://www-unix.mcs.anl.gov/otc/Guide/faq/linear-programming-faq.html>. [13]
- Lustig, I.J., Marsten, R.E., and Shanno, D.F. 1994, "Interior Point Methods: Computational State of the Art," *ORSA Journal on Computing*, vol. 6, pp. 1–14. See also Commentaries and Rejoinder, pp. 15–36. [14]
- Wright, S.J. 1997, *Primal-Dual Interior-Point Methods* (Philadelphia: S.I.A.M.). [15]
- Bixby, R.E. 2002, "Solving Real-World Linear Programs: A Decade and More of Progress," *Operations Research*, vol. 50, pp. 3–15. [16]
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.10.
- Nemhauser, G.L., Rinnooy Kan, A.H.G., and Todd, M.J. (eds.) 1989, *Optimization* (Amsterdam: North-Holland).
- Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), Chapters 7–8.
- Ignizio, J.P., and Cavalier, T.M. 1994, *Linear Programming* (Englewood Cliffs, NJ: Prentice-Hall). [Undergraduate text]
- Vanderbei, R.J. 2001, *Linear Programming: Foundations and Extensions*, 2nd ed. (Boston: Kluwer). [Undergraduate/graduate text]
- Chvátal, V. 1983, *Linear Programming* (New York: Freeman). [Undergraduate text]
- Gass, S.I. 1985, *Linear Programming: Methods and Applications*, 5th ed. (New York: McGraw-Hill), reprinted 2003 (New York: Dover). [Undergraduate text]
- Murty, K.G. 1983, *Linear Programming* (New York: Wiley). [Undergraduate text]
- Numerical Recipes Software 2007, "Routine Implementing the Simplex Method," *Numerical Recipes Webnote No. 12*, at <http://www.nr.com/webnotes?12> [17]

10.11 Linear Programming: Interior-Point Methods

As we mentioned in §10.10, the worst-case number of iterations for the simplex method is an exponential function of n . (The worst case occurs for $m = n$.) The average number of iterations, however, is a small multiple of m . For a long time it was not known if there was another algorithm for linear programming that would be bounded by, for example, some polynomial in n . In 1979, Khachian published a new algorithm [1], the *ellipsoid method*, that is in fact polynomial in n . Disappointingly, however, in practical implementations it was much slower than the simplex method.

In 1984, the field was electrified by Karmarkar's paper [2] describing an *interior-point* method. Not only was it polynomial in n , but he claimed it solved large LP problems significantly faster than the simplex method. This claim turned out to be somewhat exaggerated, but in the frenzy of activity over the next decade interior-point algorithms were developed that *do* solve many problems much faster than the simplex method, especially very large problems. Ironically, the rivalry between the two algorithms led to improvements of about a factor of 100 in the simplex method itself over the same period. We give some recommendations on which method to use at the end of this section.

Originally, interior-point methods traversed the interior of the feasible region, homing in on the optimal vertex. So-called infeasible interior-point methods follow

a path in the interior of the nonnegative region, $x_i \geq 0$, $i = 1, \dots, n$, but possibly through the infeasible region.

To understand how interior-point methods work, we need to develop some more theory about linear programming, in particular about *duality*.

10.11.1 Dual Problem

As we saw in §10.10, any LP problem can be written in standard form:

$$\begin{array}{ll} \text{minimize} & \mathbf{c} \cdot \mathbf{x} \\ \text{subject to} & \mathbf{A} \cdot \mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0 \end{array} \quad (10.11.1)$$

Here slack variables have been appended to the x 's to write all inequality constraints as equalities, but no other logical variables have been added. This is called the *primal* problem. Recall that if the constraints in (10.11.1) are satisfied, we say that $\mathbf{x}$ is a feasible point.

The *dual* problem corresponding to (10.11.1) interchanges the roles of variables and constraints: Corresponding to the m constraints is a set of variables $(y_1, \dots, y_m)$ determined by

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{c} \\ & \mathbf{y} \text{ free} \end{array} \quad (10.11.2)$$

Here “free” means unconstrained. Most of the textbooks mentioned at the end of the previous section discuss exactly how to go from the primal problem to its dual. For a hint, see §16.5.2, where a different primal-dual problem is discussed. After that, a particularly clear discussion is in [3]. Note how the constraint matrix for the dual problem is the transpose of the matrix for the primal problem. Forming the dual of the dual simply takes you back to the primal problem.

The dual problem (10.11.2) can be rewritten by adding slack variables $(z_1, \dots, z_n)$:

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} = \mathbf{c} \\ & \mathbf{z} \geq 0, \quad \mathbf{y} \text{ free} \end{array} \quad (10.11.3)$$

If $(\mathbf{y}, \mathbf{z})$ satisfy the constraints in (10.11.3), we say that they are *dual feasible*. For a good introduction to the meaning of the relation between primal and dual problems, see [4].

The *weak duality theorem* asserts that the value of the dual objective function provides a lower bound to the value of the objective function if they are each evaluated at feasible points. Proof: $\mathbf{b} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{A} \cdot \mathbf{x} = \mathbf{x} \cdot \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{x} \cdot \mathbf{c}$. The difference

$$\mathbf{c} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{y} \quad (10.11.4)$$

is called the *duality gap*. If the primal is unbounded (the objective can be made arbitrarily negative), then the dual must be infeasible, and vice versa. Moreover, we